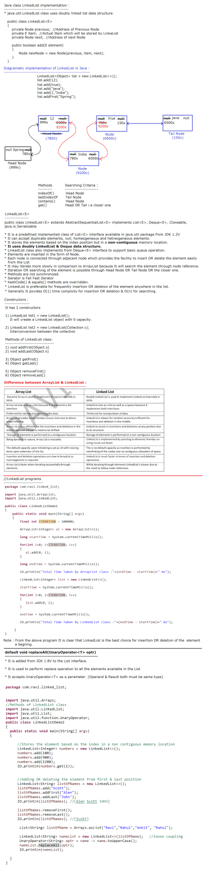

Java class LinkedList implementation :

----

* java.util.LinkedList class uses doubly linked list data structure.

```
  public class LinkedList<E>
  {
      private Node previous;  //Address of Previous Node
      private E item;  //Actual Item which will be stored by LinkeList
      private Node next;  //Address of next Node

      public boolean add(E element)
      {
           Node newNode = new Node(previous, item, next);
      }
  }
```

Diagrametic implementation of LinkedList in Java :

```
LinkedList<Object> list = new LinkedList<>();
list.add(12);
list.add(true);
list.add("java");
list.add(1,"India");
list.addFirst("Spring");
```

Methods	Searching Criteria :
indexOf()	Head Node
lastIndexOf	Tail Node
contains()	Head Node
get()	Head OR Tail i.e closer one

LinkedList<E>

public class LinkedList<E> extends AbstractSequentialList<E> implements List<E>, Deque<E>, Cloneable, java.io.Serializable

* It is a predefined implemented class of List<E> interface available in java.util package from JDK 1.2V
* It can accept duplicate elements, null, homeogeneous and hetreogeneous elements.
* It stores the elements based on the index position but in a **non-contiguous** memory location.
* **It uses doubly LinkedList & Deque data structure.**
* LinkedList class also implements from Deque<E> interface to support basic queue operation.
* Elements are inserted in the form of Node.
* Each node is connected through adjecent node which provides the facility to insert OR delete the element easily from the List
* It may iterate more slowly in comparison to ArrayList because It will search the element through node reference.
* Iteration OR searching of the element is possible through Head Node OR Tail Node OR the closer one.
* Methods are not synchronized.
* Iterator is Fail Fast Iterator
* hashCode() & equals() methods are overridden.
* LinkedList is preferable for frequently insertion OR deletion of the element anywhere in the list.
* Generally It povides O(1) time complxity for insertion OR deletion & O(n) for searching.

Constructors :

It has 2 constructors

1) LinkedList list1 = new LinkedList();
   It will create a LinkedList object with 0 capacity.

2) LinkedList list2 = new LinkedList(Collection c);
   Interconversion between the collection

Methods of LinkedList class:

1) void addFirst(Object o)
2) void addLast(Object o)

3) Object getFirst()
4) Object getLast()

5) Object removeFirst()
6) Object removeLast()

Difference between ArrayList & LinkedList :

Array List	Linked List
Dynamic Array is used to implement ArrayList internally in JAVA.	Doubly Linked List is used to implement LinkedList internally in JAVA.
Array List acts only as a list because it implements a list interface.	LinkedList acts as a list as well as a queue because it implements both interfaces.
Preferred for storing and accessing the data.	Preferred for manipulation of data.
ArrayList provides faster random access time due to direct array indexing.	LinkedList is slower for random access but efficient for insertions and deletion in the middle.
Array List is less efficient for the insertions and deletions in the middle because elements need to be shifted.	LinkedList excels in insertions and deletions at any position due to its structure.
Storage of elements is performed at a contiguous location.	Storage of elements is performed at a non-contiguous location.
Being dynamic in nature, Array List is resizable.	LinkedList is implemented by pointing to elements thereby no sizing issues are faced.
The default capacity upon initializing is set as 10 with resizing done upon extension of the list.	The is no default capacity as insertion is performed by interlinking of the nodes and no contiguous allocation of space.
Insertion and deletion operations are slow in ArrayList as rearrangement is required.	LinkedList is much faster in terms of insertion and deletion operations.
Array List is faster when iterating sequentially through elements.	While iterating through elements LinkedList is slower due to the need to follow node references.

RAVI

//LinkedList programs

```
package com.ravi.linked_list;

import java.util.ArrayList;
import java.util.LinkedList;

public class LinkedListDemo1
{
    public static void main(String[] args)
    {
        final int ITERATION = 100000;

        ArrayList<Integer> al = new ArrayList<>();

        long startTime = System.currentTimeMillis();

        for(int i=0; i<ITERATION; i++)
        {
           al.add(0, i);
        }

        long endTime = System.currentTimeMillis();

        IO.println("Total Time Taken by ArrayList class :"+(endTime - startTime)+" ms");

        LinkedList<Integer> list = new LinkedList<>();

        startTime = System.currentTimeMillis();

        for(int i=0; i<ITERATION; i++)
        {
           list.add(0, i);
        }

        endTime = System.currentTimeMillis();

        IO.println("Total Time Taken by LinkedList class :"+(endTime - startTime)+" ms");

    }
}
```

Note : From the above program It is clear that LinkedList is the best choice for insertion OR deletion of the element a begning.

---

default void replaceAll(UnaryOperator<T> optr)

* It is added from JDK 1.8V to the List interface.

* It is used to perform replace operation to all the elements available in the List

* It accepts UnaryOparator<T> as a parameter. [Operand & Result both must be same type]

```
package com.ravi.linked_list;


import java.util.Arrays;
//Methods of LinkedList class
import java.util.LinkedList;
import java.util.List;
import java.util.function.UnaryOperator;
public class LinkedListDemo2
{
  public static void main(String[] argv)
  {

     //Stores the element based on the index in a non contiguous memory location
     LinkedList<Integer> numbers = new LinkedList<>();
     numbers.add(100);
     numbers.add(900);
     numbers.add(1200);
     IO.println(numbers.get(1));


     //Adding OR deleting the element from first & last position
     LinkedList<String> listOfNames = new LinkedList<>();
     listOfNames.add("Scott");
     listOfNames.addFirst("Alen");
     listOfNames.addLast("John");
     IO.println(listOfNames); //[Alen Scott John]

     listOfNames.removeFirst();
     listOfNames.removeLast();
     IO.println(listOfNames); //[Scott]

      List<String> listOfName = Arrays.asList("Ravi","Rahul","Ankit", "Rahul");

      LinkedList<String> nameList = new LinkedList<>(listOfName);   //loose coupling
      UnaryOperator<String> optr = name -> name.toUpperCase();
      nameList.replaceAll(optr);
      IO.println(nameList);

  }
}
```